Code-Layout, Readability and Reusability

Project Name: CaptionAI: AI-powered social media caption generator

Date: 7 July 2026

Team Name: Anupriya

Maximum Marks: 5 Marks

Code Quality & Structure Details

S.No PDF	Quality Principle PDF	Implementation Details / Description PDF	Specific Project Examples (Files/Folders) PDF
1	Folder/Directory Structure	Separated frontend client application layers from the backend routing server codebase. Utilized a modular, component-based layout structure for the user interface components and an explicit route-controller architecture layout for the application API server operations.	Client root utilizes /src/components/ and /src/styles/ layouts; Server utilizes /routes/api.js and /models/UserLog.js structures.
2	Naming Conventions	Followed standardized styling constraints across layouts: camelCase for variable identifiers and function naming statements, PascalCase for independent layout structural components, and UPPER_CASE formatting keys for static environment parameters.	React component files utilize CaptionCard.jsx styling; API variables leverage generatedCaptions conventions; variables leverage .env references.
3	Code Readability & Comments	Leveraged descriptive inline comment labels preceding custom logic steps, alongside structural JSDoc header definitions explaining function behavior contexts, parameter payloads, and expected callback returns.	Used structural JSDoc blocks inside the /src/utils/openaiHandler.js logic script to document prompt interpolation steps.

S.No PDF	Quality Principle PDF	Implementation Details / Description PDF	Specific Project Examples (Files/Folders) PDF
4	Modularity & Reusability	Maintained a strict DRY (Don't Repeat Yourself) system approach by decomposing repetitive functional sections, isolated formatting loops, and structural styling styles into independent utility helper functions.	Custom TextContainer.jsx fields and CopyButton.jsx hooks are shared fluidly across dashboard screens.
5	Formatting & Linting Tools	Deployed automated formatting tools to establish uniform tab sizing gaps, punctuation configurations, and syntax parsing behaviors throughout the complete collaborative layout timeline workspace.	Configured explicit rules targeting ESLint checks and standard automated styling rules through Prettier tags in package.json.
6	Error Handling Patterns	Managed operational boundaries through descriptive try-catch processing wrappers, mapping system failure contexts down to standard HTTP error codes with unified message formats returned dynamically back to the view dashboard.	Integrated a global, centralized error catcher layer block layout inside the central server.js framework setup.